Welcome to your next lesson: Decision tables to branching logic.

I'm getting your personalized lesson ready—based on what you've learned, what's next, and how to help it stick.

Step by step, you're moving forward.

-

Imagine a movie theater that charges ticket prices based on age ?

Here's a simple rule set (we'll later turn this into **if**/elif/**else**):

- Age < 13 → child price
- Age 13–17 → teen price
- Age 18–64 → adult price
- Age 65+ → senior price

This kind of grid of rules is what we'll turn into a decision table next.

Quick check for you, just thinking it through:

- What price group should an 11-year-old get?
- What about a 70-year-old?

You can just answer with two words like: "child, senior" (or whatever you think).

child price

Nice—11 years old → child price ✅

We also had one more from that rule set:

Age 65+ → senior price

So a 70-year-old should get the senior price.

Now I want to show you how we turn a table of rules like that into **if** / elif / **else** code, step by step.

Before we do: are you comfortable with the idea that

- each row in that rule list is one case, and
- every person should match exactly one row?

Just answer yes / no, and I'll adjust how I explain it.

Yes, that makes sense

Great ?

Let's take that same rule set and turn just the first row into Python.

Rules again:

- Age < 13 → child price
- Age 13–17 → teen price
- Age 18–64 → adult price
- Age 65+ → senior price

If we only code the first row, it would look like:

Python

```
age = 10  # just an example

if age < 13:
    print("child price")
```

So:

One row in the table → one condition in code.

Does this mapping feel clear so far:

"Age < 13 → child price" becomes `if age < 13: print("child price")`?

If yes, next I'll have you help me write the row for the teen price.

Yes, that's clear

Perfect ?

Let's add the second row now:

Age 13–17 → teen price

In code, we extend the first `if` with an `elif` that matches this age range:

Python

```
age = 10  # example

if age < 13:
    print("child price")
elif age >= 13 and age <= 17:
    print("teen price")
```

Key idea:

Row 2 becomes `elif age >= 13 and age <= 17:` → `print("teen price")`.

Make sense why we used `>= 13` and `<= 17` there, instead of `<` or `>` only?

yes it akes sense

What do you think? Pick the option that best matches all ages 13 through 17 and nothing else ?

Given this rule row:

Age 13–17 → teen price

Which of these is the best condition to use for that row in Python, assuming there is already an earlier row handling `age < 13`?

A.

`age > 13 and age < 17`

B.

`age >= 13 and age <= 17`

C.

age > 13 and age <= 17

D.

age >= 13 and age < 17

Nice work on that one ?

age >= 13 and age <= 17 is perfect because:

- age >= 13 includes 13 (doesn't skip it)
- age <= 17 includes 17
- Together, they cover exactly 13, 14, 15, 16, 17 — and nothing else
- And since an earlier row already handles age < 13, there's no overlap.

Next step, we'll turn the adult and senior rows into code and then check that our whole chain is both exclusive (no overlaps) and complete (every age fits somewhere).

Ready to build out the rest of that if/elif/else chain in the code editor next?

Yes, I'm ready

Awesome ?

Let's turn the whole decision table into code now.

Rules again:

- Age < 13 → child price
- Age 13–17 → teen price
- Age 18–64 → adult price
- Age 65+ → senior price

Step by step, your if chain should look like this shape:

Python

```
age = 10  # you can change this to test

if age < 13:
    print("child price")
elif age >= 13 and age <= 17:
    print("teen price")
elif age >= 18 and age <= 64:
    print("adult price")
else:
    print("senior price")
```

Your turn ?

1. Open the code editor.
2. Type that code (don't copy–paste; typing helps it stick).
3. Change age a few times (like 10, 15, 40, 70) and hit Run to see which message prints.
4. When it looks good, hit Submit code so I can check it.

Python

```
age = 99

if age < 13:
    print("child price")
elif age >= 13 and age <= 17:
    print("teen price")
elif age >= 18 and age <= 64:
    print("adult price")
else:
    print("senior price")
```

Nice job, Ron ? Your code runs correctly, and age = 99 going to "senior price" is exactly right.

You've now turned the whole decision table into an if/elif/else chain.

Next idea: we need to check two things about this chain:

1. Mutual exclusivity – for any single age, only one branch can ever run.
2. Completeness – every possible age is handled by some branch.

Let's check exclusivity first, with a simple mind-check:

Look at just these two rows:

- if age < 13:
- elif age >= 13 and age <= 17:

Is there any age that could make both of those conditions true at the same time, or are they non-overlapping?

You can answer in your own words, like "they can't overlap because …" or "they might overlap at …".

non overlapping first covers 0 - 12 second covers 13 to 17 exlusively

Exactly right ?

- age < 13 → 0 through 12
- age >= 13 and age <= 17 → 13 through 17

There's no age that fits both at once, so those two rows are mutually exclusive.

Now let's test completeness of the whole chain.

Look at your four branches:

- < 13
- 13–17
- 18–64
- else (senior)

Think about all ages from 0 up to something like 120. Do you think there is any age that would not match any of those branches, or are we complete now?

Just answer: “complete” or “missing something (and what)”.

it looks complee as far as whole number go

Yep, nice catch: for whole-number ages, your table is complete ✅

Quick recap of what you just did:

- Turned a 4-row decision table into clean `if` / `elif` / `else` code
- Checked mutual exclusivity (no overlapping age ranges)
- Checked completeness (every integer age lands somewhere)

One more small step: I want to be sure the “first true wins” behavior is solid in your mind, since that matters a lot with decision tables.

I’ll give you a quick multiple-choice question next to check that—sound good?

Sounds good!

What do you think this one does? Take your time with the question below ?

Consider this code:

```
age = 15

if age < 18:
    print("minor")
elif age < 21:
    print("young adult")
else:
    print("adult")
```

What will this program print?

A.

minor

B.

young adult

C.

adult

D.

It will print both "minor" and "young adult"

minor as 15 < 18 and it exit the if eli chain on firt mtch

Exactly ?

For `age = 15` in that question:

- The condition `age < 18` is True, so it prints `"minor"`

- And because **if**/elif/**else** is first true wins, Python never even checks the later branch.

You've clearly got how the chain flows now ?

Next, I'd like to show you a small decision table and have you help me turn each row into conditions (no code yet, just the logic). Up for one more like that, or do you want to pause here and ask anything about decision tables / exclusivity / completeness first?

I'm ready for another table

Nice, let's build another one ?

Here's a shipping fee decision table for an online store:

Order total	Shipping fee
< $25	$10
$25–$49.99	$5
≥ $50	Free

We'll later turn this into code, but for now we just want the conditions in plain logic.

Step by step, starting with the first row:

Order total < $25 → shipping fee $10

In Python-style logic, that condition would be:

total < 25

So row 1 condition is: total < 25

Your turn ?

What would be the condition for the second row ("$25–$49.99 → $5") written as a logical test, like we've been doing?

You can answer in text, e.g. total >= 25 and total < 50.

order total >= $25 and <= $49.99 -> shipping fee $5

Perfect logic ?

Written in Python-style, that's:

Python
```
total >= 25 and total < 50
```

That matches $25 up to (but not including) $50, which is exactly $25–$49.99.

So far we have:

- Row 1: total < 25 → $10
- Row 2: total >= 25 and total < 50 → $5

Notice how those two don't overlap and there's no gap between 24.99 and 25.

Last row is:

≥ $50 → Free

To keep everything exclusive and complete, which condition do you think is best for that row?

You can pick one of these worded options:

- A: `total > 50`
- B: `total >= 50`
- C: `total <= 50`

Just answer A, B, or C.

B

Nice, B is exactly what we want ?

`total >= 50`:

- Includes 50 (so $50 gets free shipping)
- Covers everything above 50
- Doesn't overlap with the earlier row (`total < 50`)

So the full table as conditions is:

- `total < 25` → $10
- `total >= 25 and total < 50` → $5
- `total >= 50` → Free

You've now:

- Read a decision table
- Turned each row into clean, non-overlapping conditions
- Thought about completeness (all totals are covered)

Next step: want to turn this shipping table into actual Python `if`/elif/`else` code in the editor, like we did with ages?

Yes, let's try it

Great ?□˙ŧ□

Here's the decision table again:

Order total	Shipping fee
< 25	10
25–49.99	5
≥ 50	0 (free)

Let's follow the same shape as the age example. Your code should look like:

- `if total < 25: print(10)`
- `elif total >= 25 and total < 50: print(5)`
- `else: print(0)` (covers `total >= 50`)

Your turn in the code editor:

1. Create a variable, e.g. `total = 37.5` (you can change this to test).

2. Write an `if` / elif / `else` chain that:

- prints `10` when `total < 25`
- prints `5` when `25 <= total < 50`
- prints `0` when `total >= 50`

3. Hit Run to test a few totals (like 20, 25, 49.99, 50).
4. When it looks right, hit Submit code, and I'll check it.

```python
total = 66
if total < 25:
    print("Shipping fee is $10")
elif total >= 25 and total <= 50:
    print("Shipping fee is $5")
elif total > 50:
    print("Shipping fee is free")
```

Nice work, Ron ?

Your logic is almost perfect and your output for `total = 66` is correct:

"Shipping fee is free" ✅

There's just one small thing I want you to notice: overlap between rows.

Your conditions:

```python
if total < 25:
    print("Shipping fee is $10")
elif total >= 25 and total <= 50:
    print("Shipping fee is $5")
elif total > 50:
    print("Shipping fee is free")
```

Look at the second and third lines:

- `elif total >= 25 and total <= 50`
- `elif total > 50`

Do they overlap? No. But do they leave any gap? Also no.

So this chain is exclusive and complete for all totals ✅

We could also write the last part as just `else` since anything not `< 25` and not between `25` and `50` must be `> 50`.

Quick check for understanding:

If I change your code so the middle line is:

```python
elif total >= 25 and total < 50:
```

and the last line is:

```python
else:
    print("Shipping fee is free")
```

Would that behave exactly the same for all order totals, or would it change anything?

Answer in your own words: “same behavior” or “different because …”.

if tthe total is exactly 50 it would have printed shipping is $5 now it woud print Shipping is free What was the almost part of my loic the not just uig else as an endig clause to the elif>?

You’re exactly right about the behavior change ?

- Before: `total <= 50` → $5, so 50 gets $5
- After: `total < 50` and `else` → free, so 50 gets free shipping

So yes: that change does change what happens at 50.

About the “almost” part:

Your logic is correct and complete. The “almost” was just about a cleaner style, not about being wrong.

- Your version (totally fine):

```python
elif total > 50:
    print("Shipping fee is free")
```

- A common refactor:

```python
else:
    print("Shipping fee is free")
```

Why `else` can work here:

- First row handles: `total < 25`
- Second row handles: `25 <= total <= 50`
- So anything that reaches the last branch must be `total > 50`

→ `else` is guaranteed to mean “greater than 50” here.

So: your logic is solid; the “almost” was just that we could simplify the last condition to `else` in this specific table.

Want to try one more tiny table where you write the `if`/`elif`/`else` yourself from scratch, or would you rather review how to check for overlaps and gaps in a table?

Let's review overlaps and gaps

Great choice ? Let’s keep it visual and simple.

Think of number lines for overlaps and gaps.

1. Overlap (bad for exclusivity)

Example:

```python
```

```python
if age < 18:
    print("minor")
elif age <= 21:
    print("young adult")
```

On a number line:

- age < 18 → ... 15, 16, 17
- age <= 21 → ... 17, 18, 19, 20, 21

The number 17 is in both ranges → that's an overlap.

If it were a decision table, two rows would both be "true" for 17 (fortunately in code, only the first one actually runs, but the table itself isn't clean).

2. Gap (bad for completeness)

Example:

Python
```python
if age < 18:
    print("minor")
elif age > 21:
    print("adult")
```

On a number line:

- age < 18 → ... 15, 16, 17
- age > 21 → 22, 23, ...

Ages 18, 19, 20, 21 are in neither range → that's a gap.

Quick check for you ?

Look at this pair:

Python
```python
if score < 60:
    print("fail")
elif score >= 60 and score <= 89:
    print("pass")
```

Just thinking about whole-number scores 0–100:

- Do you see any overlap between these two ranges?
- Do you see any gap (numbers not covered)?

You can answer like: "no overlap, no gap" or "gap at …" etc.

no overlap but 90-100 not ocoverd

Exactly right ✅

For

Python
```python
if score < 60:
    print("fail")
```

```
elif score >= 60 and score <= 89:
    print("pass")
```

- No overlap:
- First: 0–59
- Second: 60–89
- Nothing is in both.
- Gap:
- Scores 90–100 are not covered at all → gap in the decision table.

So:

- Overlap = some values hit two rows
- Gap = some values hit no rows

Next, I'd like to have you decide if a small 3-row table has an overlap, a gap, both, or neither using a quick multiple-choice. Ready for that?

I'm ready!

What do you think this table has—overlap, gap, both, or neither? Take your time and choose an option below ?

Look at this decision table for exam results (0–100 whole-number scores):

- Row 1: score < 60 → "fail"
- Row 2: score >= 60 and score <= 80 → "pass"
- Row 3: score > 80 → "honors"

Thinking about all scores from 0 to 100, what does this table have?

A.

An overlap between some rows

B.

A gap (some scores are not covered)

C.

Both an overlap and a gap

D.

No overlap and no gap

Nice, nailed it ?

For that table:

- Row 1: score < 60 → 0–59
- Row 2: **score** >= 60 **and score** <= 80 → 60–80
- Row 3: score > 80 → 81–100

So:

- No overlap: no score is in two rows at once

- No gap: every score 0–100 fits some row
  That's exactly what we want from a clean decision table ✅
  You've now:
- Turned 3–4 row tables into `if`/elif/`else`
- Checked for overlaps and gaps
- Understood "first true wins" in the chain
  Last thing to lock in: I'll give you one more small scenario, and you tell me which input would fall into the `else` branch. That'll tie together completeness and branch ordering. Ready for that final quick check?

Yes, let's do it!

What do you think? Pick the age that would hit the else branch ?

Given this Python code:

```
age = 12

if age < 5:
    print("toddler")
elif age >= 5 and age <= 12:
    print("child")
elif age >= 13 and age <= 17:
    print("teen")
else:
    print("adult")
```

Which single value of age below would make the program print "adult" (i.e., hit the `else` branch)?

A.
12
B.
17
C.
18
D.
5

Nice finish, Ron ?

You chose 18, which is exactly right:

- age < 5 → toddler
- 5–12 → child
- 13–17 → teen

- Anything not caught by those (like 18, 30, 80) falls into the **else** → "adult"
  So 18 skips all earlier branches and lands in **else**.
  You've shown you can:
- Translate 3–5 row decision tables into clean **if**/elif/**else**
- Check for mutual exclusivity (no overlaps)
- Check for completeness (no gaps; else or full coverage)
  You've mastered everything for this lesson ?
  Do you feel ready to wrap up this lesson and move on to the next one, or do you want to practice one more custom table first?
  I'm ready to move on
  Awesome work today, Ron ?
  You:
- Took real-world rule tables (ages, shipping, scores) and turned them into clean **if**/elif/**else** chains
- Checked mutual exclusivity by spotting overlaps in ranges
- Checked completeness by finding gaps and using **else** or full numeric coverage
  Those are exactly the skills you'll use any time someone hands you a requirements table in the real world.
  As you head into the next lesson, keep this mental checklist handy:

1. Turn each row → condition
2. Scan for overlaps
3. Scan for gaps (and maybe use **else**)
   If anything in the next lesson feels shaky or connects back to decisions, tell me and we'll plug the gap right away ?